

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

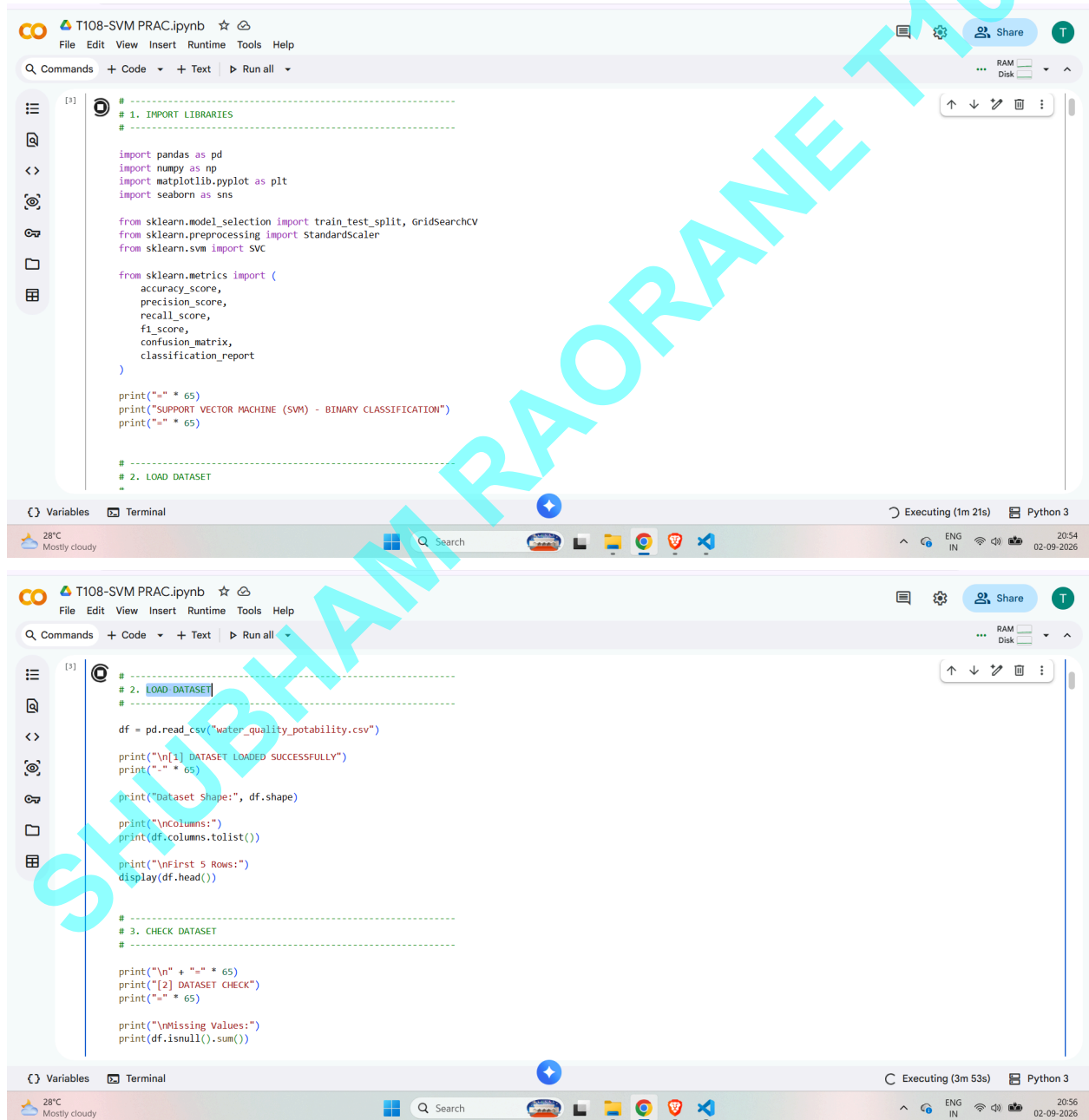
AIM : Support Vector Machines (SVM)

Implement the SVM algorithm for binary classification.

Train an SVM model using a given dataset and optimize its parameters.

Evaluate the performance of the SVM model on test data and analyze the results.

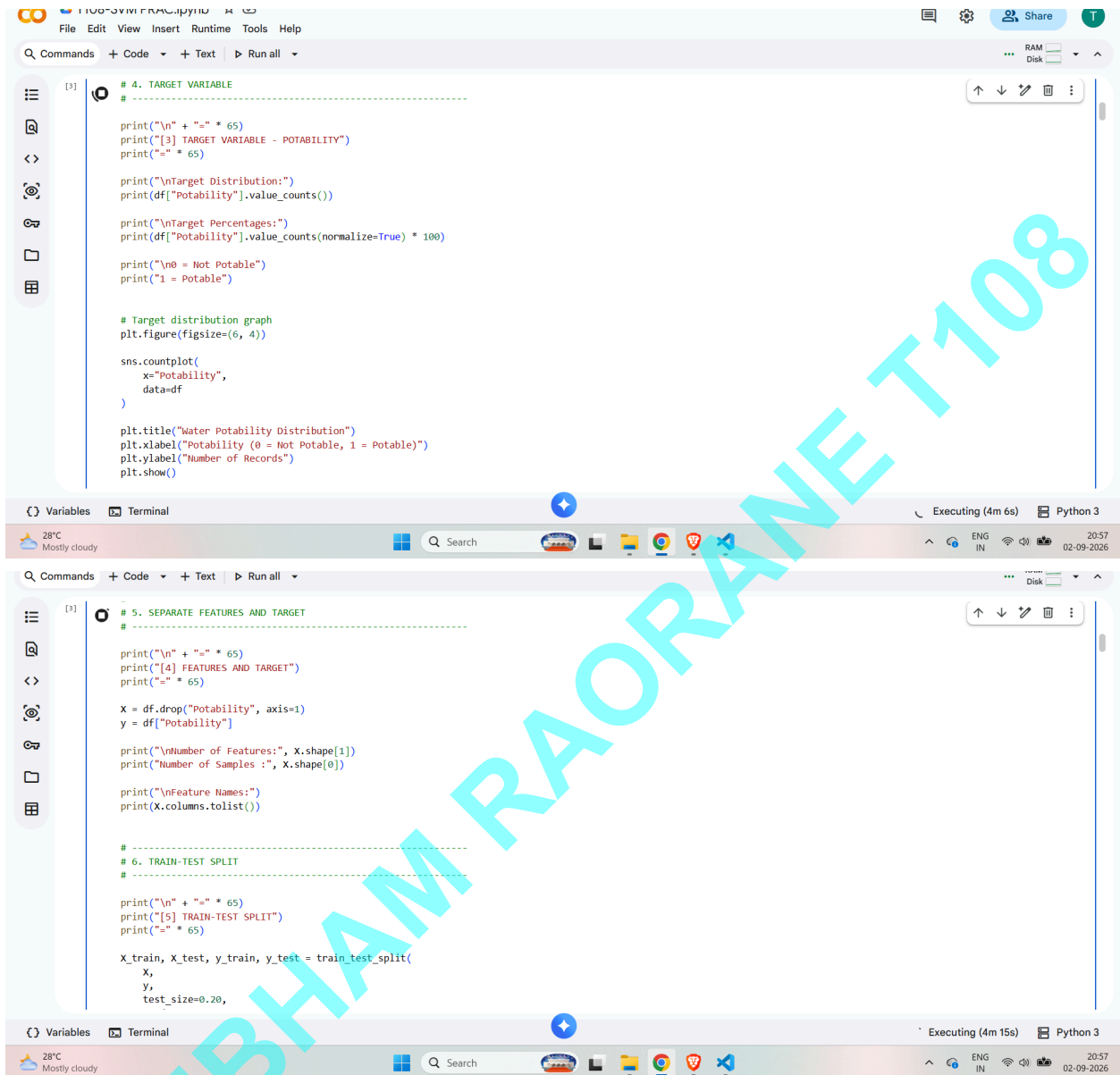
CODE:



```
# -----  
# 1. IMPORT LIBRARIES  
# -----  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split, GridSearchCV  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
  
from sklearn.metrics import (   
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix,  
    classification_report  
)  
  
print("=" * 65)  
print("SUPPORT VECTOR MACHINE (SVM) - BINARY CLASSIFICATION")  
print("=" * 65)  
  
# -----  
# 2. LOAD DATASET  
# -----  
  
df = pd.read_csv("water_quality_potability.csv")  
  
print("\n[1] DATASET LOADED SUCCESSFULLY")  
print("-" * 65)  
  
print("Dataset Shape:", df.shape)  
  
print("\nColumns:")  
print(df.columns.tolist())  
  
print("\nFirst 5 Rows:")  
display(df.head())  
  
# -----  
# 3. CHECK DATASET  
# -----  
  
print("\n" + "=" * 65)  
print("[2] DATASET CHECK")  
print("=" * 65)  
  
print("\nMissing Values:")  
print(df.isnull().sum())
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



The image displays two screenshots of a Jupyter Notebook interface, likely Google Colab, showing Python code for data analysis and machine learning preparation. A large, diagonal watermark "SHUBHAM RAORANE T108" is overlaid across the center of the image.

Top Screenshot: # 4. TARGET VARIABLE

```
# 4. TARGET VARIABLE
# -----

print("\n" + "=" * 65)
print("[3] TARGET VARIABLE - POTABILITY")
print("=" * 65)

print("\nTarget Distribution:")
print(df["Potability"].value_counts())

print("\nTarget Percentages:")
print(df["Potability"].value_counts(normalize=True) * 100)

print("\n0 = Not Potable")
print("1 = Potable")

# Target distribution graph
plt.figure(figsize=(6, 4))

sns.countplot(
    x="Potability",
    data=df
)

plt.title("Water Potability Distribution")
plt.xlabel("Potability (0 = Not Potable, 1 = Potable)")
plt.ylabel("Number of Records")
plt.show()
```

Bottom Screenshot: # 5. SEPARATE FEATURES AND TARGET

```
# 5. SEPARATE FEATURES AND TARGET
# -----

print("\n" + "=" * 65)
print("[4] FEATURES AND TARGET")
print("=" * 65)

X = df.drop("Potability", axis=1)
y = df["Potability"]

print("\nNumber of Features:", X.shape[1])
print("Number of Samples :", X.shape[0])

print("\nFeature Names:")
print(X.columns.tolist())

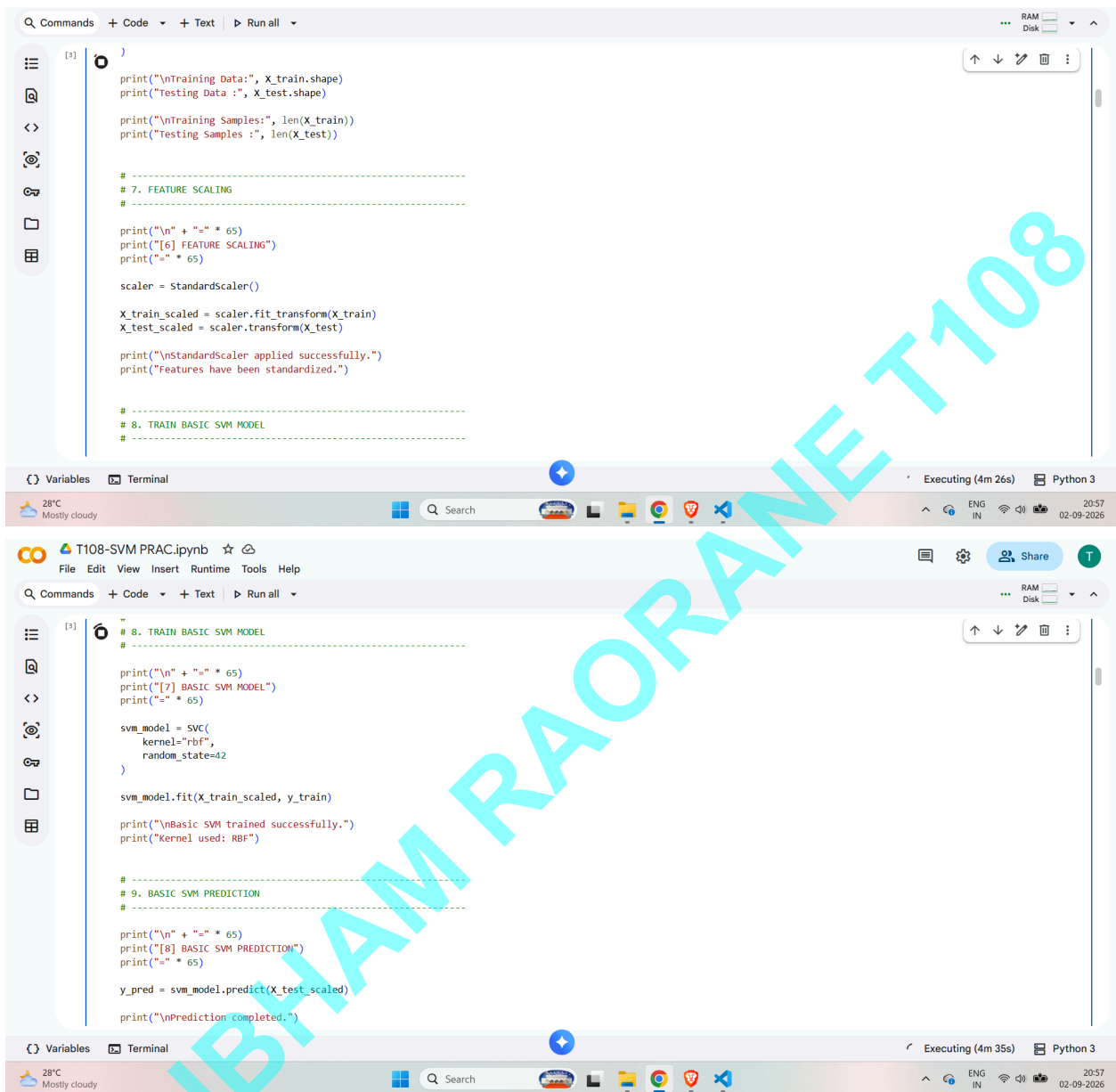
# -----
# 6. TRAIN-TEST SPLIT
# -----

print("\n" + "=" * 65)
print("[5] TRAIN-TEST SPLIT")
print("=" * 65)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



The image displays two sequential screenshots of a Jupyter Notebook interface, showing the implementation of a Support Vector Machine (SVM) model. A large, diagonal watermark reading "SHUBHAM RAORANE T108" is overlaid across the center of the page.

Top Screenshot: This screenshot shows the initial steps of the SVM implementation. The code includes:

- Printing the shapes of training and testing data.
- Printing the number of samples in the training and testing sets.
- A section header: `# 7. FEATURE SCALING`.
- Printing the current index (6) and the number of features (65).
- Creating a `StandardScaler` object.
- Fitting the scaler on the training data and transforming both training and testing data.
- Printing a success message for the scaler and a note that features have been standardized.
- A section header: `# 8. TRAIN BASIC SVM MODEL`.

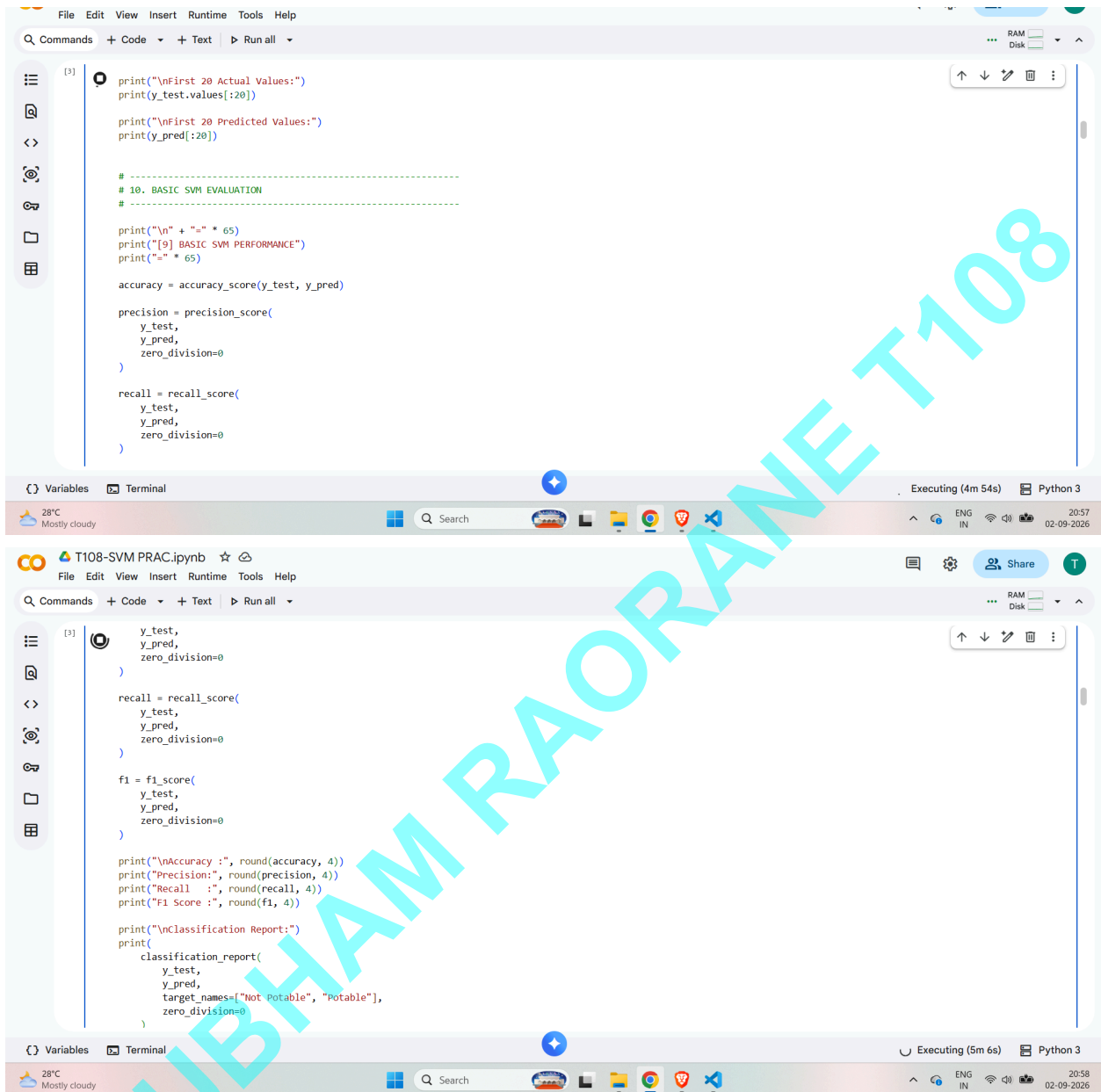
Bottom Screenshot: This screenshot shows the continuation of the SVM implementation. The code includes:

- Printing the current index (7) and the number of features (65).
- Creating an `SVC` object with `kernel='rbf'` and `random_state=42`.
- Fitting the SVM model on the scaled training data.
- Printing a success message for the SVM training and the kernel used (RBF).
- A section header: `# 9. BASIC SVM PREDICTION`.
- Printing the current index (8) and the number of features (65).
- Using the trained model to predict on the scaled testing data.
- Printing a completion message for the prediction.

Both screenshots show the notebook running on a Python 3 environment, with a terminal window at the bottom and a status bar indicating the execution time (4m 26s for the top screenshot and 4m 35s for the bottom screenshot).

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



```
[3] print("\nFirst 20 Actual Values:")
print(y_test.values[:20])

print("\nFirst 20 Predicted Values:")
print(y_pred[:20])

# -----
# 10. BASIC SVM EVALUATION
# -----

print("\n" + "=" * 65)
print("[9] BASIC SVM PERFORMANCE")
print("=" * 65)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

y_test,
y_pred,
zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\nAccuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Not Potable", "Potable"],
        zero_division=0
    )
)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
# -----  
# 11. BASIC SVM CONFUSION MATRIX  
# -----  
  
print("\n" + "=" * 65)  
print("[10] BASIC SVM CONFUSION MATRIX")  
print("=" * 65)  
  
cm = confusion_matrix(y_test, y_pred)  
  
print("\nConfusion Matrix:")  
print(cm)  
  
plt.figure(figsize=(6, 5))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=["Not Potable", "Potable"],  
    yticklabels=["Not Potable", "Potable"]  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Basic SVM Confusion Matrix")  
  
plt.show()
```

```
# -----  
# 12. PARAMETER OPTIMIZATION USING GRIDSEARCHCV  
# -----  
  
print("\n" + "=" * 65)  
print("[11] SVM PARAMETER OPTIMIZATION")  
print("=" * 65)  
  
print("\nGridSearchCV is testing different SVM parameters.")  
print("Please wait...")  
  
param_grid = {  
    "C": [0.1, 1, 10, 100],  
    "kernel": ["linear", "rbf", "poly"],  
    "gamma": ["scale", "auto"]  
}  
  
grid_search = GridSearchCV(  
    estimator=SVC(random_state=42),  
    param_grid=param_grid,  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)  
  
grid_search.fit(X_train_scaled, y_train)  
  
print("\nParameter optimization completed successfully!")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
# 13. BEST PARAMETERS
# -----
print("\n" + "=" * 65)
print("[12] BEST SVM PARAMETERS")
print("=" * 65)

print("\nBest Parameters:")

for parameter, value in grid_search.best_params_.items():
    print(parameter, ":", value)

print("\nBest Cross-Validation Accuracy:")
print(round(grid_search.best_score_, 4))

# -----
# 14. OPTIMIZED SVM MODEL
# -----
print("\n" + "=" * 65)
print("[13] OPTIMIZED SVM MODEL")
print("=" * 65)

best_svm = grid_search.best_estimator_

print("\nOptimized SVM:")
print(best_svm)

# -----
# 15. OPTIMIZED SVM PREDICTIONS
# -----
print("\n" + "=" * 65)
print("[14] OPTIMIZED SVM PREDICTION")
print("=" * 65)

y_pred_best = best_svm.predict(X_test_scaled)

print("\nOptimized prediction completed.")

print("\nFirst 20 Actual Values:")
print(y_test.values[:20])

print("\nFirst 20 Optimized Predictions:")
print(y_pred_best[:20])

# -----
# 16. OPTIMIZED SVM PERFORMANCE
# -----
print("\n" + "=" * 65)
print("[15] OPTIMIZED SVM PERFORMANCE")
print("=" * 65)

accuracy_best = accuracy_score(
    y_test,
    y_pred_best
)

precision_best = precision_score(
    y_test,
    y_pred_best,
    zero_division=0
)

recall_best = recall_score(
    y_test,
    y_pred_best,
    zero_division=0
)

f1_best = f1_score(
    y_test,
    y_pred_best,
    zero_division=0
)

print("\nAccuracy :", round(accuracy_best, 4))
print("Precision:", round(precision_best, 4))
print("Recall:", round(recall_best, 4))
print("F1 Score:", round(f1_best, 4))
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```

f1_best = f1_score(
    y_test,
    y_pred_best,
    zero_division=0
)

print("\nAccuracy :", round(accuracy_best, 4))
print("Precision:", round(precision_best, 4))
print("Recall   :", round(recall_best, 4))
print("F1 Score :", round(f1_best, 4))

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred_best,
        target_names=["Not Potable", "Potable"],
        zero_division=0
    )
)

```

```

[3]: print("\n" + "=" * 65)
      print("[16] OPTIMIZED SVM CONFUSION MATRIX")
      print("=" * 65)

      cm_best = confusion_matrix(
          y_test,
          y_pred_best
      )

      print("\nConfusion Matrix:")
      print(cm_best)

      plt.figure(figsize=(6, 5))

      sns.heatmap(
          cm_best,
          annot=True,
          fmt="d",
          cmap="Blues",
          xticklabels=["Not Potable", "Potable"],
          yticklabels=["Not Potable", "Potable"]
      )

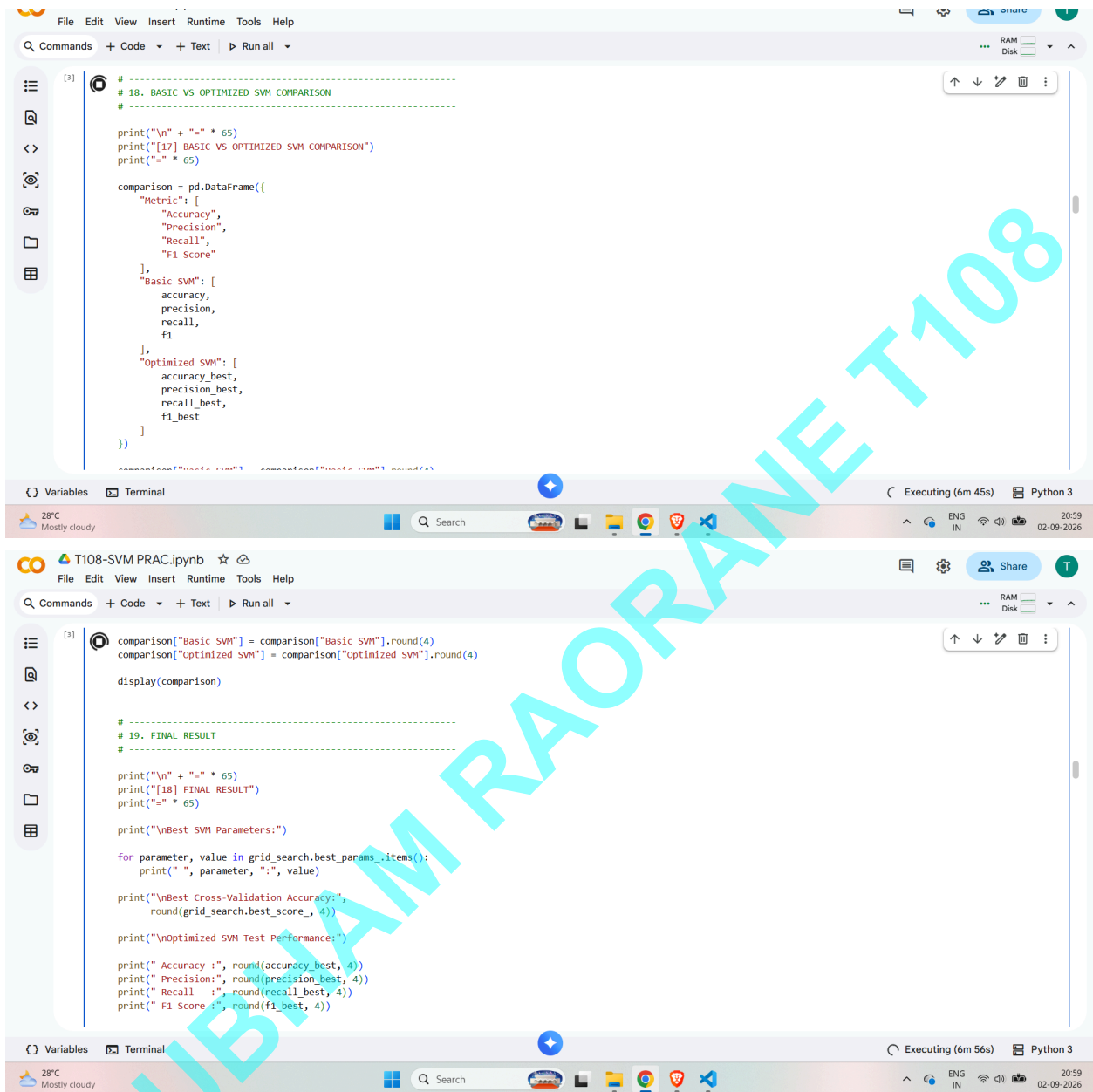
      plt.xlabel("Predicted")
      plt.ylabel("Actual")
      plt.title("Optimized SVM Confusion Matrix")

      plt.show()

```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



The image displays two screenshots of a Jupyter Notebook interface, likely from a web browser. The top screenshot shows a code cell with the following Python code:

```
[3]: # -----  
# 18. BASIC VS OPTIMIZED SVM COMPARISON  
# -----  
  
print("\n" + "=" * 65)  
print("[17] BASIC VS OPTIMIZED SVM COMPARISON")  
print("=" * 65)  
  
comparison = pd.DataFrame(  
    "Metric": [  
        "Accuracy",  
        "Precision",  
        "Recall",  
        "F1 Score"  
    ],  
    "Basic SVM": [  
        accuracy,  
        precision,  
        recall,  
        f1  
    ],  
    "Optimized SVM": [  
        accuracy_best,  
        precision_best,  
        recall_best,  
        f1_best  
    ]  
)  
  
comparison["Basic SVM"] = comparison["Basic SVM"].round(4)  
comparison["Optimized SVM"] = comparison["Optimized SVM"].round(4)
```

The bottom screenshot shows the same notebook with the following code:

```
[3]: comparison["Basic SVM"] = comparison["Basic SVM"].round(4)  
comparison["Optimized SVM"] = comparison["Optimized SVM"].round(4)  
  
display(comparison)  
  
# -----  
# 19. FINAL RESULT  
# -----  
  
print("\n" + "=" * 65)  
print("[18] FINAL RESULT")  
print("=" * 65)  
  
print("\nBest SVM Parameters:")  
  
for parameter, value in grid_search.best_params_.items():  
    print(" ", parameter, ":", value)  
  
print("\nBest Cross-Validation Accuracy:",  
      round(grid_search.best_score_, 4))  
  
print("\nOptimized SVM Test Performance:")  
  
print(" Accuracy :", round(accuracy_best, 4))  
print(" Precision:", round(precision_best, 4))  
print(" Recall   :", round(recall_best, 4))  
print(" F1 Score  :", round(f1_best, 4))
```

Both screenshots show a status bar at the bottom indicating "Executing (6m 45s)" and "Python 3". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a search bar, and a sidebar with icons for file management and execution.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

OUTPUT

```
CO T108-SVM PKAL.ipynb File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
RAM
Disk

=====
SUPPORT VECTOR MACHINE (SVM) - BINARY CLASSIFICATION
=====

[1] DATASET LOADED SUCCESSFULLY
=====
Dataset Shape: (10000, 10)

Columns:
['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity', 'Organic_carbon', 'Trihalomethanes', 'Turbidity', 'Potability']

First 5 Rows:

```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
0	7.160466	195.712338	22636.785632	7.235746	336.794067	424.396711	14.376178	68.105888	3.991681	1
1	7.187738	193.202237	21263.028076	7.117409	333.997161	430.163405	14.206928	64.807703	3.957064	0
2	5.400302	140.739062	17266.593422	10.056852	328.358241	472.874073	11.256381	56.931906	4.824786	0
3	7.078913	196.886558	21539.185679	7.104867	335.223406	429.400333	14.344261	65.289765	3.909317	0
4	7.052283	194.971523	21659.468727	7.101790	335.669903	425.620822	14.100794	66.178504	3.989340	0

```
=====
[2] DATASET CHECK
=====

Missing Values:
ph          0
Hardness    0
Solids      0
Chloramines 0
Sulfate     0
Conductivity 0
Organic_carbon 0
Trihalomethanes 0
Turbidity   0
Potability  0

dtypes: float64(9), int64(1)
memory usage: 781.4 KB

=====
[3] TARGET VARIABLE - POTABILITY
=====

Target Distribution:
Potability
1    5000
0    5000
Name: count, dtype: int64

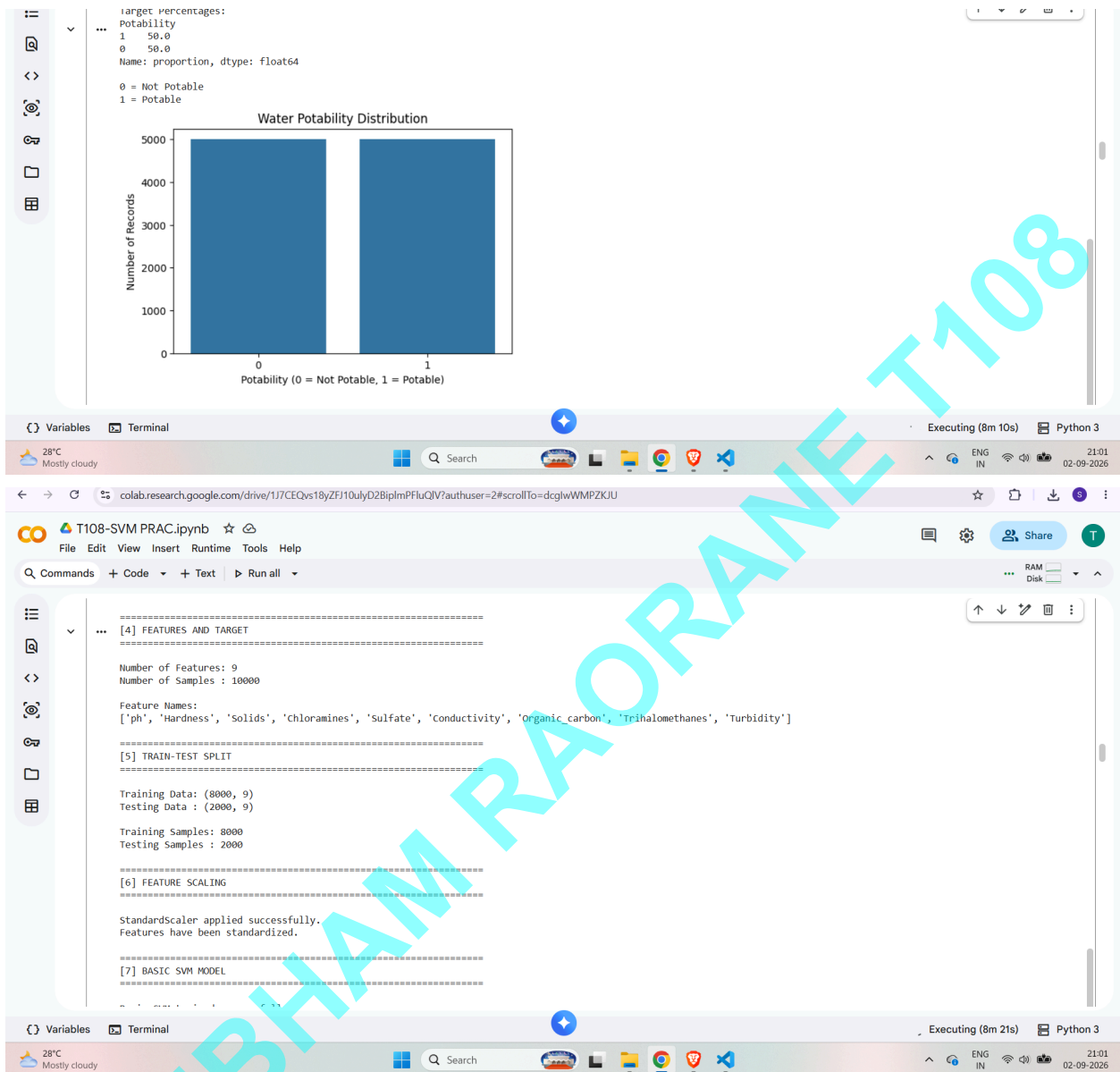
Target Percentages:
Potability
1     50.0
0     50.0
Name: proportion, dtype: float64

0 = Not Potable
1 = Potable

Water Potability Distribution
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

The screenshot displays a Jupyter Notebook interface with two cells. The top cell, titled '[7] BASIC SVM MODEL', shows the successful training of an SVM model using the RBF kernel. The bottom cell, titled '[9] BASIC SVM PERFORMANCE', displays the model's performance metrics and a confusion matrix.

[7] BASIC SVM MODEL

Basic SVM trained successfully.
Kernel used: RBF

[8] BASIC SVM PREDICTION

Prediction completed.

First 20 Actual Values:
[0 1 0 1 0 0 0 1 0 1 0 1 1 1 1 1 1 0 0]

First 20 Predicted Values:
[0 1 0 1 0 0 0 1 0 1 0 1 1 1 1 1 1 0 0]

[9] BASIC SVM PERFORMANCE

Accuracy : 0.8525
Precision: 0.8762
Recall : 0.821
F1 Score : 0.8477

Classification Report:

	precision	recall	f1-score	support
Not Potable	0.83	0.88	0.86	1000
Potable	0.88	0.82	0.85	1000

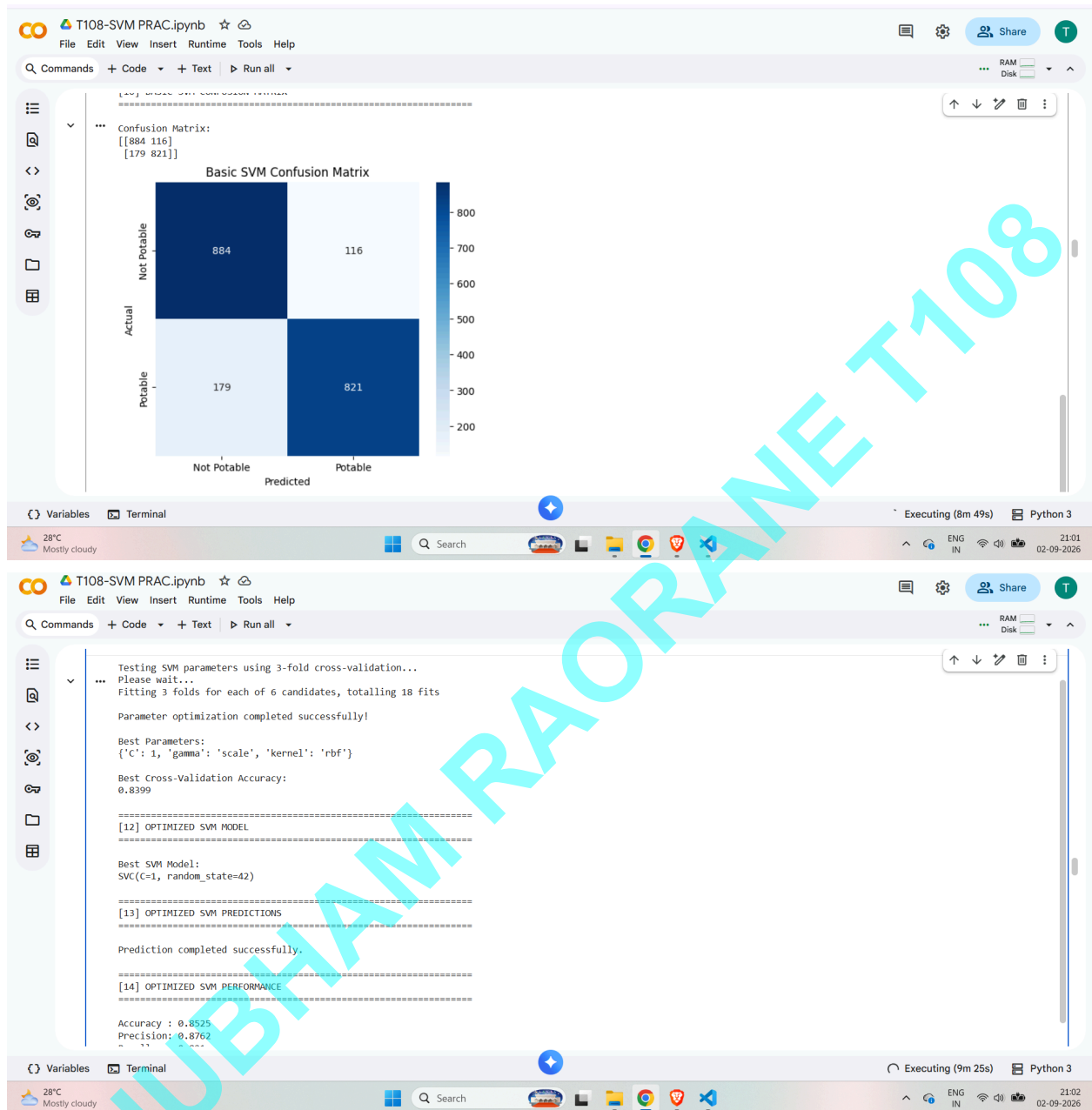
[10] BASIC SVM CONFUSION MATRIX

Confusion Matrix:
[[884 116]
[179 821]]

Basic SVM Confusion Matrix

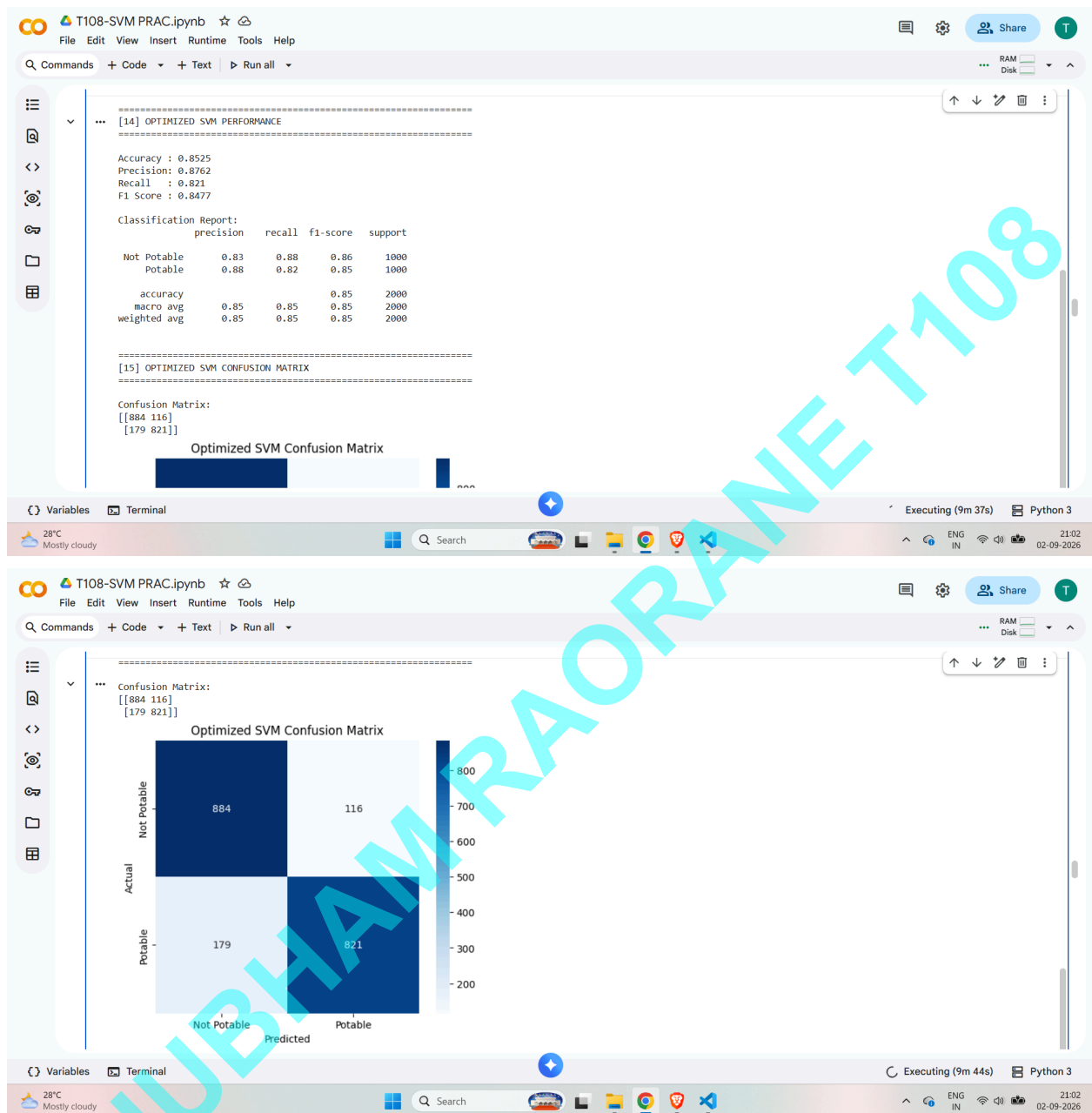
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



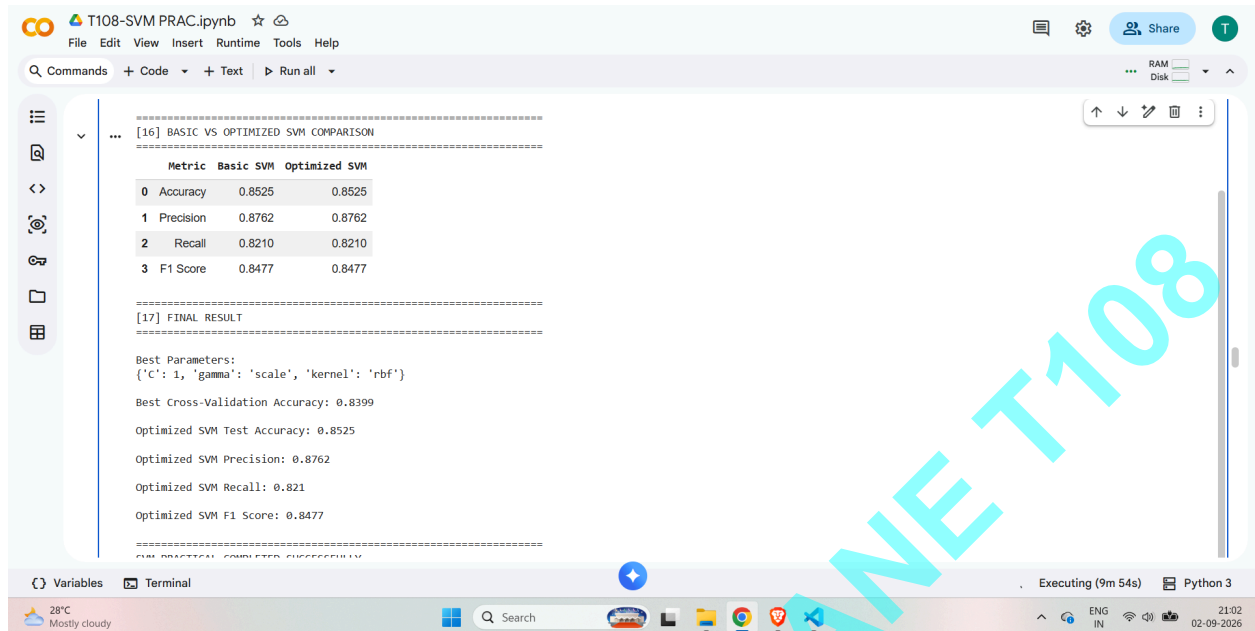
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI



The screenshot displays a Jupyter Notebook titled "T108-SVM PRAC.ipynb". The notebook content includes a comparison of Basic vs. Optimized SVM performance metrics and final results.

[16] BASIC VS OPTIMIZED SVM COMPARISON

	Metric	Basic SVM	Optimized SVM
0	Accuracy	0.8525	0.8525
1	Precision	0.8762	0.8762
2	Recall	0.8210	0.8210
3	F1 Score	0.8477	0.8477

[17] FINAL RESULT

Best Parameters:
{ 'C': 1, 'gamma': 'scale', 'kernel': 'rbf' }

Best Cross-Validation Accuracy: 0.8399

Optimized SVM Test Accuracy: 0.8525

Optimized SVM Precision: 0.8762

Optimized SVM Recall: 0.821

Optimized SVM F1 Score: 0.8477

SHUBHAM RAORANE T108